A COMPARATIVE LOOK AT
PROGRAM VALIDATION

Software Testing and Formal Verification

How we gain confidence that software behaves correctly — through execution, and through proof.

PROGRAM VALIDATION

Testing & Formal Verification

Testing

- Validates by running the program with selected inputs
- Checks observed behavior against expected results

Formal Verification

- Validates by mathematical proof of correctness
- Ensures the program always satisfies the specification

Both are validation techniques — one empirical, one formal. Used together, they provide stronger assurance than either alone.

FUNDAMENTALS

Testing

What it is

- Execution of a program (or component) with selected inputs
- Aim: detect bugs, not prove correctness

Strengths

- Widely used, intuitive, relatively low cost
- Can reveal unexpected behaviors
- Works well for large and complex systems

Limitations

- Cannot exhaustively cover all inputs / states
- Only shows the presence of bugs, never their absence
- Quality depends on test coverage and scenarios chosen

Formal Verification

What it is

- Mathematical proof that a system meets a specification
- Uses logic, model checking, or theorem proving

Strengths

- Provides high assurance of correctness
- Covers all possible inputs / states (within the formal model)
- Detects corner cases beyond human intuition

Limitations

- Requires a precise formal specification
- High expertise and computational cost
- Less practical for very large, evolving systems

HEAD TO HEAD

Testing vs. Formal Verification

Aspect	Testing	Formal Verification
Goal	Find bugs	Prove correctness
Approach	Run the system with selected inputs	Mathematical proof against the spec
Coverage	Partial (sampled inputs / states)	Complete (all possible cases)
Cost & effort	Lower, scalable	Higher, expertise-intensive
Practicality	Excellent for large, evolving systems	Best for critical, safety/security domains
Guarantee	Bugs found ≠ correctness	Strong correctness assurance

Model Based Testing

> *Program testing can be used to show the presence of bugs, but never to show their absence.*
>
> — Edsger W. Dijkstra

- However, testing does not require a formal specification (model).
- Formal verification is used in particular for safety-critical systems.
- **When a model exists ⇒ Model Based Testing**
- **Model Based Testing ⇒ Test automation**

Test Generation



What it takes

- Testing is execution of a program (or component) with selected inputs
- Exhaustive testing is usually not possible
- Test generation is a (the) major part of software testing



Test Cases

- Scenarios to be checked



Test Data (Suites)

- Actual values used in testing

AUTOMATION

Test Generation & Automation



Execution

- Automated test execution is relatively easy



Generation

- Test generation is a difficult problem
- Automated generation is much more complicated



With a Model

- The existence of a formal model helps a great deal

IN PRACTICE

Main Test Generation Strategies



White Box (Structural)

- Test values are derived from the implementation (code)



Black Box (Functional)

- Test values are derived from requirements (specification)

Grey box strategies also exist — combining elements of both.

WORKED EXAMPLE

The Triangle Program



The program accepts three positive integers as the lengths of a triangle's sides, and classifies the triangle into one of four groups:

Equilateral

All sides have equal lengths

`return 1`

Isosceles

Two sides equal, but not all three

`return 2`

Scalene

All lengths are unequal

`return 3`

Impossible

Cannot form a triangle, or forms only a flat line

`return 4`

Adapted from cs.bris.ac.uk (COMS12100 course materials); appears in Myers' "The Art of Software Testing."

BLACK BOX

Functional Testing

- Divide the domain into equivalence classes and choose at least one value per class

Equilateral	Isosceles	Scalene	Impossible

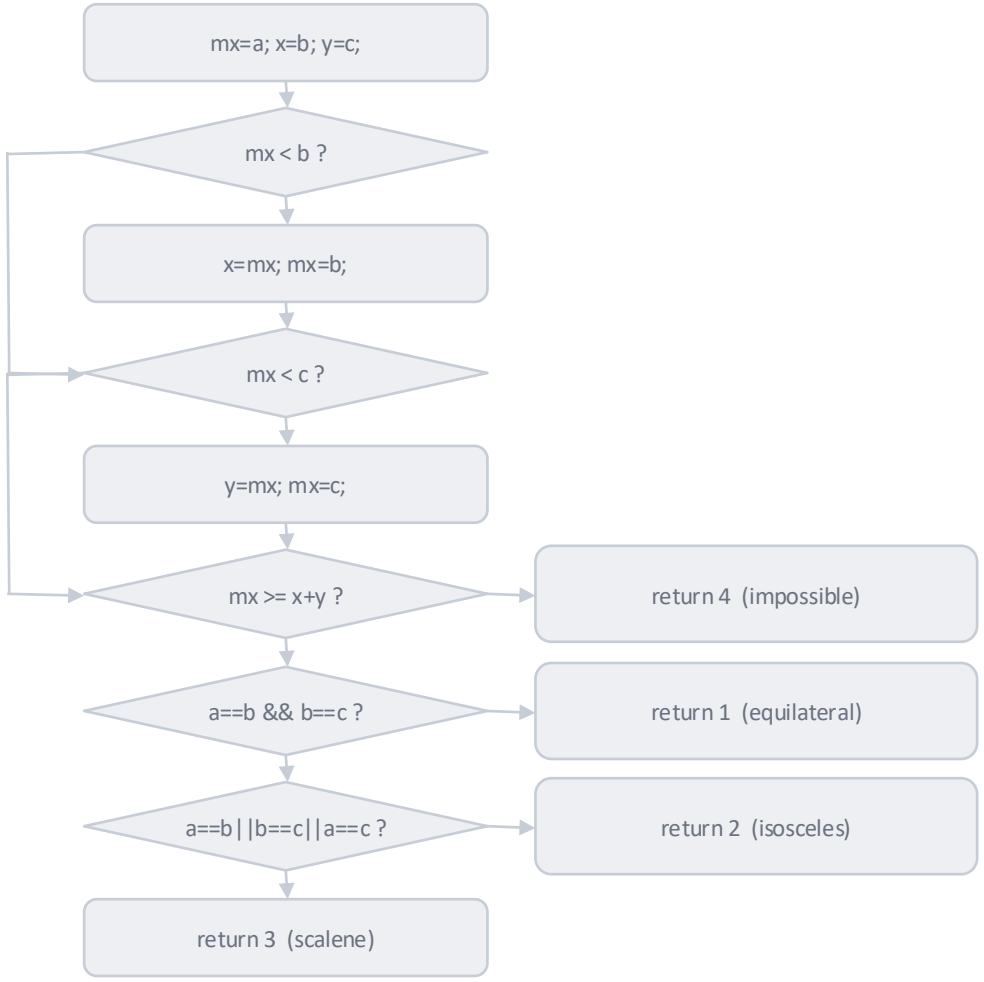


Boundary value analysis: choose values near the boundaries of each class — errors tend to lurk near boundaries.

Structural Testing

- Represent the program as a directed graph and select test values to cover:
  - Nodes (statements)
  - Branches (jump from one statement to another)
  - More complex structures (sequences, conditions, etc.)

The triangle program's control-flow graph →



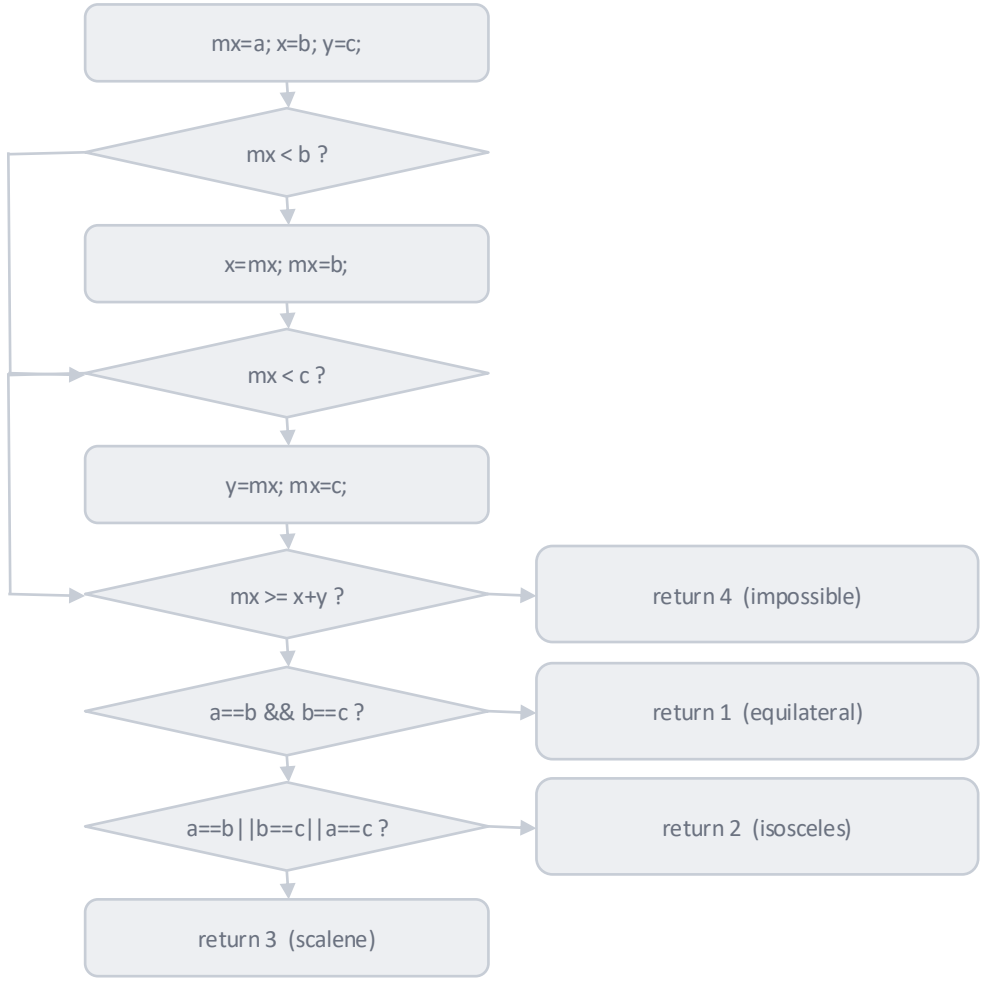
Every diamond is a decision; every path is a potential test.

WORKED EXAMPLE

Java Implementation

```
int triangle(int a, int b, int c) {
  int mx, x, y;
  mx = a; x = b; y = c;
  if (mx < b) {
    x = mx; mx = b;
  }
  if (mx < c) {
    y = mx; mx = c;
  }
  if (mx >= x + y) {
    return 4; // impossible
  }
  if (a == b && b == c) {
    return 1; // equilateral
  }
  if (a == b || b == c || a == c) {
    return 2; // isosceles
  }
  return 3; // scalene
}
```

Test case: (a, b, c) = (3, 5, 3)

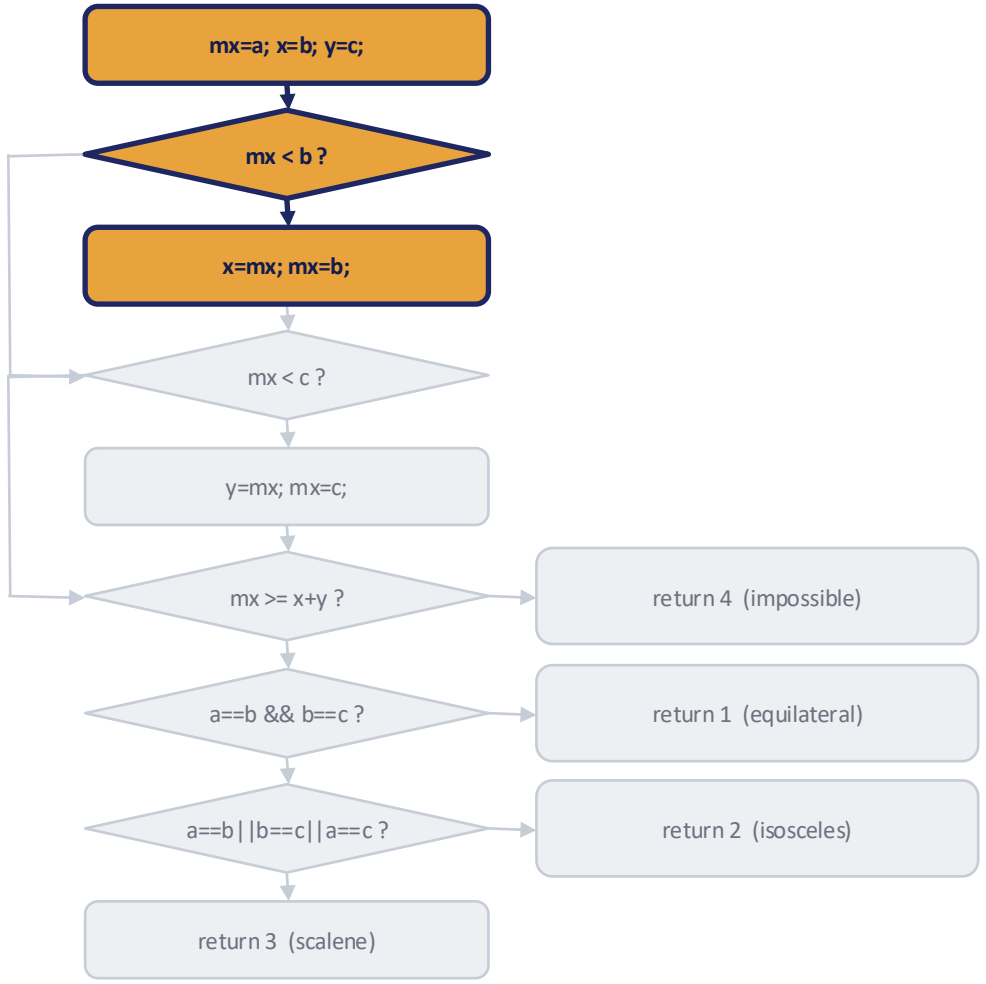


solid = executed for this test · gray = branch not exercised

Tracing Test Case (3, 5, 3)

```
int triangle(int a, int b, int c) {
  int mx, x, y;
  mx = a; x = b; y = c;
  if (mx < b) {
    x = mx; mx = b;
  }
  if (mx < c) {
    y = mx; mx = c;
  }
  if (mx >= x + y) {
    return 4; // impossible
  }
  if (a == b && b == c) {
    return 1; // equilateral
  }
  if (a == b || b == c || a == c) {
    return 2; // isosceles
  }
  return 3; // scalene
}
```

Assign mx, x, y — then check mx < b (true) and swap

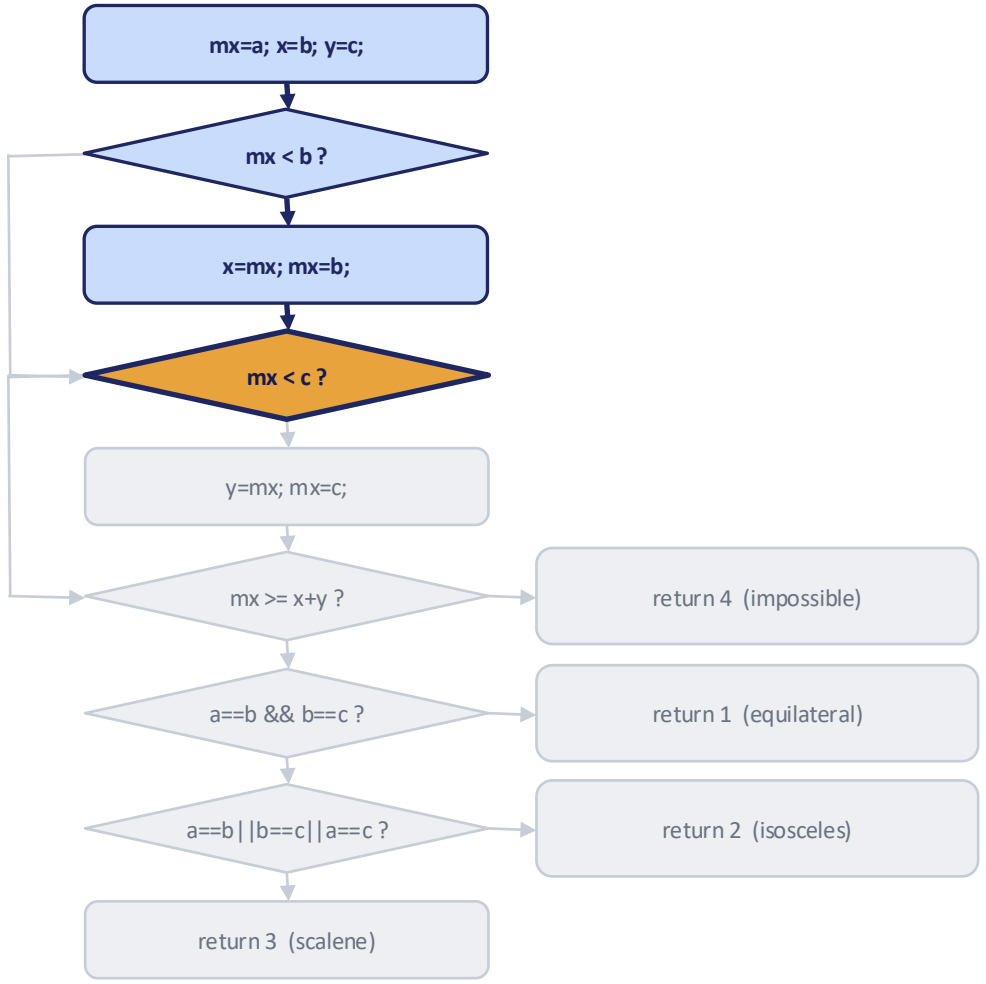


solid = executed for this test · gray = branch not exercised

Tracing Test Case (3, 5, 3)

```
int triangle(int a, int b, int c) {
  int mx, x, y;
  mx = a; x = b; y = c;
  if (mx < b) {
    x = mx; mx = b;
  }
  if (mx < c) {
    y = mx; mx = c;
  }
  if (mx >= x + y) {
    return 4; // impossible
  }
  if (a == b && b == c) {
    return 1; // equilateral
  }
  if (a == b || b == c || a == c) {
    return 2; // isosceles
  }
  return 3; // scalene
}
```

Check mx < c — false, this branch is skipped

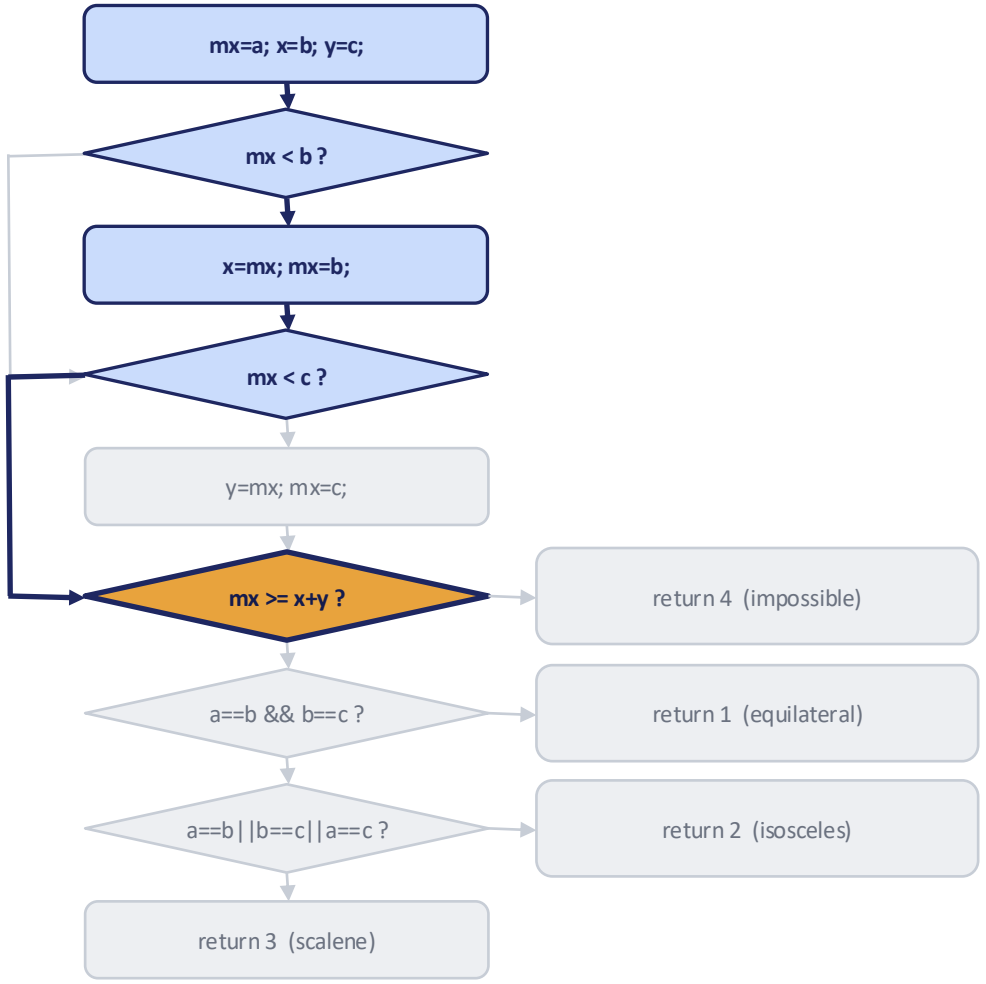


solid = executed for this test · gray = branch not exercised

Tracing Test Case (3, 5, 3)

```
int triangle(int a, int b, int c) {
  int mx, x, y;
  mx = a; x = b; y = c;
  if (mx < b) {
    x = mx; mx = b;
  }
  if (mx < c) {
    y = mx; mx = c;
  }
  if (mx >= x + y) {
    return 4; // impossible
  }
  if (a == b && b == c) {
    return 1; // equilateral
  }
  if (a == b || b == c || a == c) {
    return 2; // isosceles
  }
  return 3; // scalene
}
```

Check mx ≥ x + y — false, not impossible

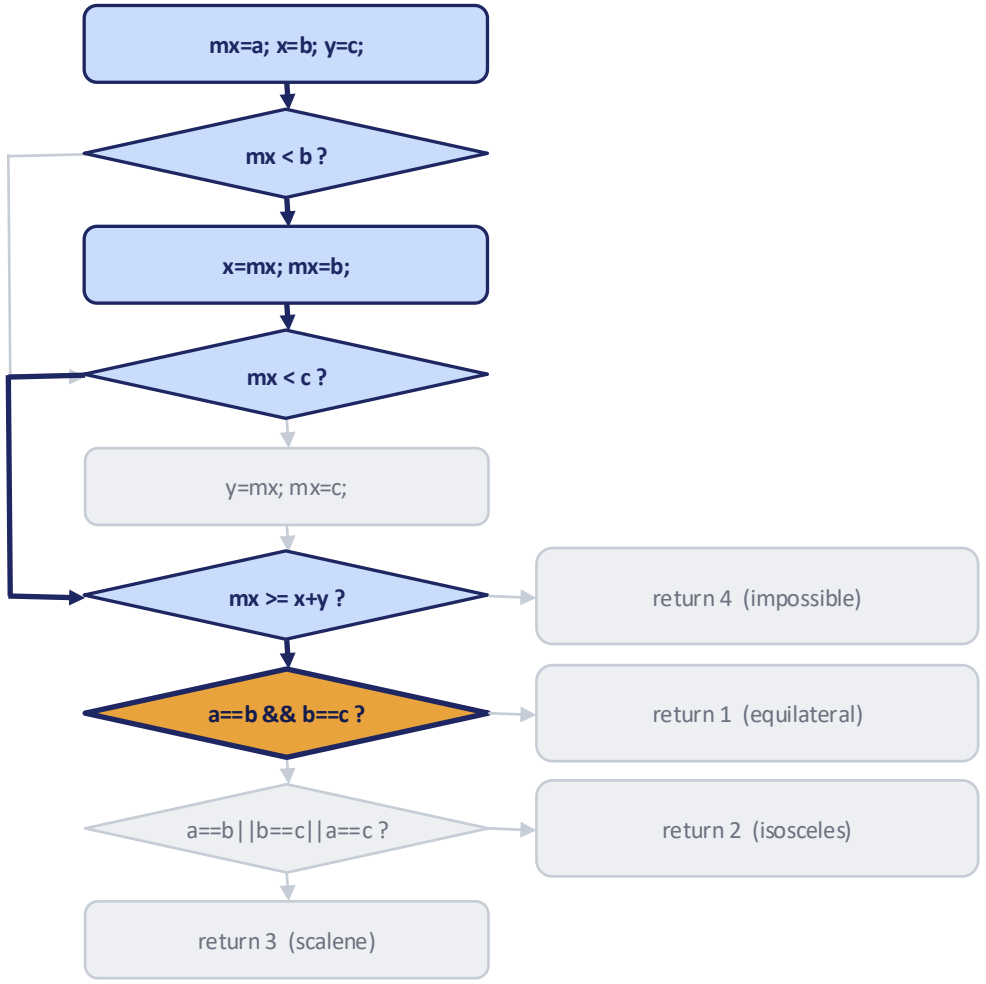


solid = executed for this test · gray = branch not exercised

Tracing Test Case (3, 5, 3)

```
int triangle(int a, int b, int c) {
  int mx, x, y;
  mx = a; x = b; y = c;
  if (mx < b) {
    x = mx; mx = b;
  }
  if (mx < c) {
    y = mx; mx = c;
  }
  if (mx >= x + y) {
    return 4; // impossible
  }
  if (a == b && b == c) {
    return 1; // equilateral
  }
  if (a == b || b == c || a == c) {
    return 2; // isosceles
  }
  return 3; // scalene
}
```

Check a==b && b==c — false, not equilateral

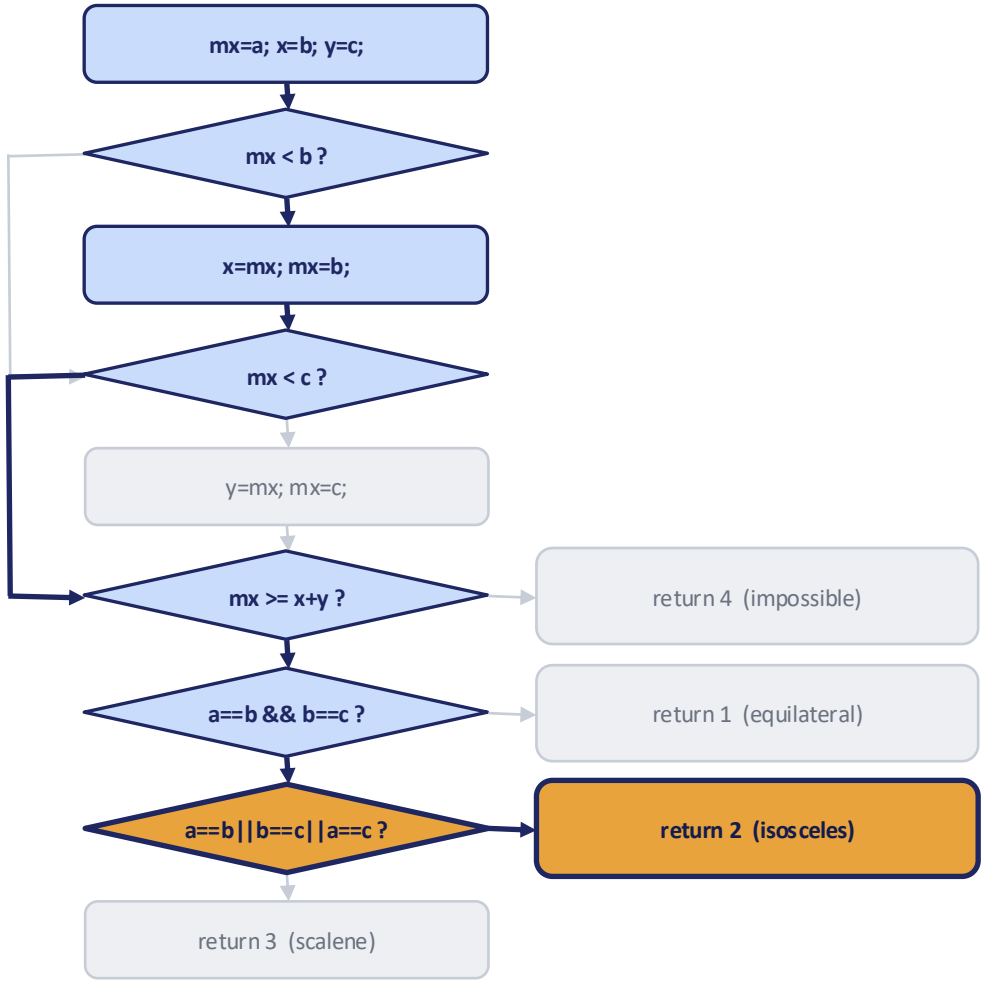


solid = executed for this test · gray = branch not exercised

Tracing Test Case (3, 5, 3)

```
int triangle(int a, int b, int c) {
  int mx, x, y;
  mx = a; x = b; y = c;
  if (mx < b) {
    x = mx; mx = b;
  }
  if (mx < c) {
    y = mx; mx = c;
  }
  if (mx >= x + y) {
    return 4; // impossible
  }
  if (a == b && b == c) {
    return 1; // equilateral
  }
  if (a == b || b == c || a == c) {
    return 2; // isosceles
  }
  return 3; // scalene
}
```

a==b || b==c || a==c is true → returns 2 (isosceles)



solid = executed for this test · gray = branch not exercised

Test Data Selection

- Structural testing selects paths through the program (the directed graph)
- But how do we choose the actual test values to exercise these paths?



From paths to values

- Framed as a **constraint satisfaction problem**
- When the system of constraints is complex**: Search Based Software Testing**

Search Based Software Testing

The idea

- Uses a meta-heuristic optimizing search technique to automate a testing task — e.g. automatic test data generation
- Used when the search space is very large and the problem is complex

Local Search

- e.g. hill-climbing, simulated annealing

Global Search

- e.g. genetic algorithms

Key: *a problem-specific fitness function guides the search toward good solutions within a practical time limit.*

Mutation Testing (Mutation Analysis)

What it is

- Primarily a method of assessing the quality of already-generated test data
- A method of structural software testing that modifies a program's source code in small ways
- **The modified source code is called a mutant.**

Mutant Construction

ORIGINAL

```
if (a && b)
  c = 0;
else
  c = 1;
```

MUTANT 1

```
if (a || b)
  c = 0;
else
  c = 1;
```

MUTANT 2

```
if (a && b)
  c = 1;
else
  c = 1;
```

MUTANT 3

```
if (a && b)
  c = 0;
else
  c = 0;
```

Each mutant flips one operator or constant — a single, small syntactic change from the original.

Mutation Operators

- Mutation “operators” are ways of introducing errors into correct code — many have been defined and implemented

Traditional Mutants

- Classic operators covering common code constructs

Specialized Environments

- e.g. object-oriented languages, via tools such as MuJava

Quality of Tests

- If the test suite detects the change (one of the tests fails), the mutant is said to be killed.

Mutation Score $MS = D / (L + D)$

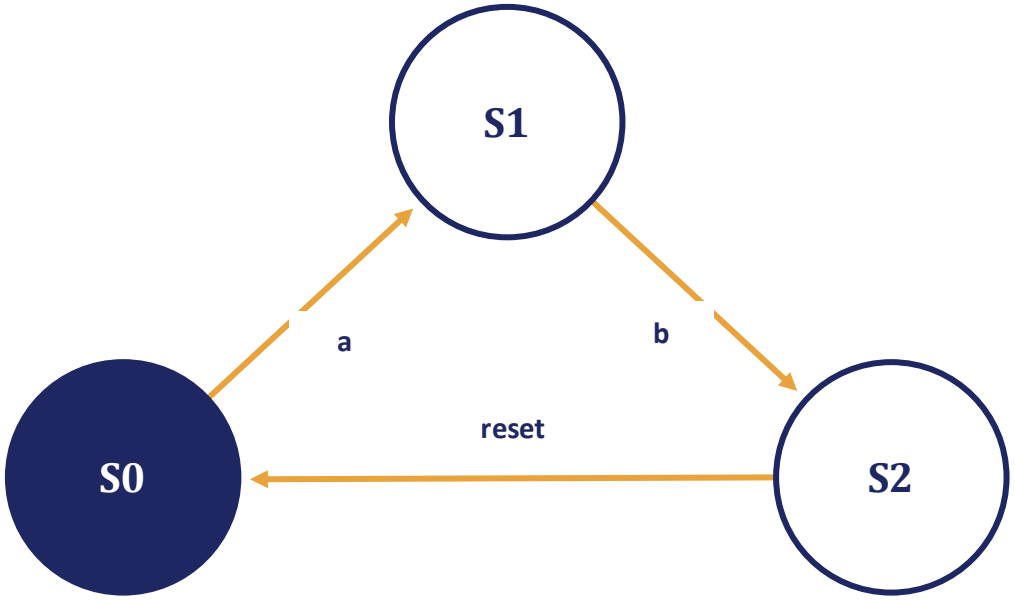
D = dead mutants **L** = live mutants

Main problem: equivalent mutants

- Equivalent mutants are functionally equivalent to the original program — they can never be killed
- Detecting equivalent mutants is a manual, very complex task

Model Based Testing

- Typically, state-transition models are used (FSM, EFSM, Statecharts)
- Test generation can be automated
- Various coverage criteria are used
- **More powerful FSM-based methods (e.g. the W-method) can even guarantee some kind of fault detection**



Illustrative finite-state model — states and labeled transitions

Formal Verification

Definition

- The process of checking whether a system's design meets specific requirements (properties)

Hoare Logic

- Reasoning about program correctness via pre- and post-conditions

Refinement-Based

- Proving correctness by stepwise refinement from abstract to concrete models

Model Checking

- Verifying a finite-state model against a spec (commonly LTL or CTL)

Requires significant time and specialized expertise. Widely used in safety-critical domains — aerospace, automotive, medical devices.

Hoare Logic

What it is

- A formal system, introduced by C.A.R. Hoare, for reasoning about program correctness
- Correctness is expressed with a precondition and postcondition placed around a command

Hoare Triple $\{ P \}\ C\ \{ Q \}$

P = precondition
C = command
Q = postcondition

Assignment

Substitutes the assigned expression into the postcondition

Sequence

The postcondition of the first command becomes the precondition of the next

Conditional

Proves the then- and else-branches separately, under the same postcondition

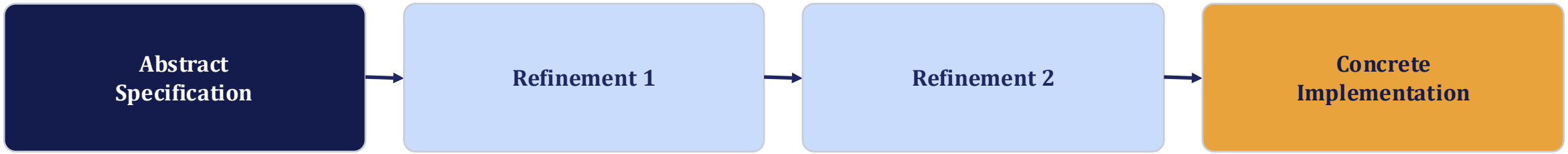
Loop

Requires a loop invariant — true before and after every iteration

Refinement-Based Verification

What it is

- Correctness by construction: start from an abstract specification and refine it stepwise into concrete, executable code
- Each refinement step must preserve the properties already proven at the more abstract level



Used in formal methods such as the B-Method, Event-B, and Z — each step is checked against the previous one, not against the final code alone.

Model Checking

What it is

- Automated, exhaustive verification of a finite-state model against a specification
- Properties are expressed in temporal logic — commonly LTL or CTL



Strength: fully automated, and a failed check returns a concrete counterexample. **Limitation:** state-space explosion for large or infinite-state systems.

Conclusions

- Testing and formal verification are complementary validation techniques — one empirical, one mathematical
- Model-Based Testing bridges the two, enabling automated test generation from a formal model
- White-box, black-box, and grey-box strategies derive tests differently — but no single test case covers every path, as the triangle-program trace showed
- Search-Based Software Testing and mutation testing help automate test generation and assess test-suite quality, respectively
- Formal methods (Hoare logic, refinement, model checking) give strong guarantees testing alone cannot — at a higher cost, best reserved for safety-critical systems



No single technique guarantees a bug-free system. Combining broad, low-cost testing with targeted formal guarantees, where the stakes justify it, gives the strongest assurance of correctness.